Validated Quantum Algorithm Benchmarks of the Metatron QSO Simulation Framework

Technical Note, version 2 — corrects and replaces version 1 of 26 November 2025

Sebastian Klemm

QSO 1.1.0 · 2026-10-24

Abstract. This note reports benchmarks of Metatron QSO 1.1.0, an open-source quantum simulation framework written in Rust with Python bindings. It covers the categories of version 1 of this note — Grover and multi-target Grover search, continuous-time quantum walks on Metatron's Cube, QAOA, VQE, variational classifiers, quantum machine learning on graphs, boson sampling and a comparison with other frameworks — and replaces all of its results, which are withdrawn (Section 1). Every number here is computed by code in the public repository and compared with an independent reference: a closed form, a published value, or a second implementation. Simulated Grover success probabilities match $\sin^2((2k+1)\theta)$ to 8.9×10^{-13}; the time-averaged walk on Metatron's Cube converges to the exact value 145/169 at the start vertex; QAOA reproduces the proven optima on rings and the published values 0.6925 (p = 1) and 0.7559 (p = 2) on large-girth cubic graphs; VQE reaches the full-CI energy of H_2 to 4.4×10^{-16} Ha; the exact boson sampler passes a χ^2 test against exactly computed output distributions. State vectors agree with Qiskit Aer, Qulacs, Cirq and PennyLane Lightning to within 2.4×10^{-15}.

1 Correction of version 1

> Version 1 of this note (26 November 2025) reported benchmark results of QSO 0.1. **All of its quantitative results are withdrawn.** An audit of the QSO 0.1 code (published with the repository as `docs/AUDIT_v0.1.md`) showed that the "quantum system" was a single 13-level system rather than a register of qubits, that several algorithms were implemented incorrectly, and that some reported numbers were constants written into the code rather than measurements. Version 1 was described as "CI-validated"; the continuous-integration checks only verified that result files existed. The author regrets the error.

Table 1 lists every claim of version 1 and the reason it is withdrawn. QSO was subsequently rewritten from scratch (version 1.0, October 2026); the results below are produced by the rewritten code and supersede version 1 completely.

Table 1. Results of version 1 and their status.

claim in version 1 (26 November 2025)	status	reason
Grover: success 0.0754, "speed-up 7.21"	withdrawn	13-level system without amplification (0.0754 ≈ 1/13); 13/1.803 compares unrelated quantities
Multi-target Grover: success 0.459	withdrawn	same engine; no reference value
Boson sampling: visibility 0.75, correlation 0.65	withdrawn	multi-photon path returned an error; both values were literals in the code
QML on graphs: test accuracy 0.40	withdrawn	not reproducible; below chance for two classes
Cross-system scores 0.937 vs 0.735–0.777	withdrawn	scores of Qiskit, Cirq, PennyLane and ProjectQ were hard-coded constants
Execution times 0.189 ms vs 1100–1250 ms	withdrawn	baseline times were constants; QSO times refer to a 13 × 13 system
Integration: fidelity 0.95, ratio 1.0, speed-up 14.42	withdrawn	derived from the items above
QAOA: ratio 1.0 on 3–5-node graphs	withdrawn	cost operator was −L/2 on a 13-level system, evolution used the non-unitary I − iHt
Walks: stationary peak 0.774, mixing ≈ 19	withdrawn	degenerate eigenspaces ignored; the correct limiting value is 145/169 ≈ 0.858

claim in version 1 (26 November 2025)	status	reason
Walks: quantum 4.15 vs classical 12 steps, speed-up 2.89	withdrawn	probabilities at successive times treated as independent measurements; the classical value 12 is correct
VQC: train and test accuracy 1.0	withdrawn	no dataset or split recorded; not reproducible
VQE: errors 0.013 / 0.000 / 0.001	withdrawn	13 × 13 Hamiltonian with trivial spectrum {0, −13}; the “Metatron ansatz” no longer exists
“CI-validated”	withdrawn	the CI only checked that JSON files existed; the comparison script accepted unknown metrics

2 Scope and methods

Software. QSO 1.1.0 simulates n-qubit pure states (up to 34 qubits) and density matrices (up to 14 qubits) with multi-threaded kernels, computes exact gradients by adjoint differentiation [17], and provides the algorithms benchmarked here. Conventions: qubit 0 is the least significant bit of a basis index, time evolution is e^{-iHt} and rotations are $R_P(\theta) = e^{-i\theta P/2}$. All random numbers come from one seeded generator (xoshiro256++), and parallel sums are combined in a fixed order, so every result is bit-for-bit reproducible on the same platform for any number of threads.

Validation principle. A benchmark value is reported only together with an independent reference: a closed-form result, a value published in the literature, an exact computation by a different method (for example Lanczos diagonalisation for VQE), or a different software package. The test-suite of the repository (177 Rust tests and 182 Python tests, run on Linux, macOS and Windows for every change) checks the same references; this note collects them with their numerical agreement.

Reproducibility. All numbers are produced by two programs in the repository and stored as JSON next to this note:

```
cargo run --release --example technical_note -- --out technical_note.json
python benchmarks/cross_framework/run.py --out cross_framework.json
```

The data in this version were computed from commit `a2675e2e1b186684d361b5313691c5c0d53c3c09` on a cloud VM, 4 vCPU Intel Xeon @ 2.10 GHz (AVX-512), 15 GB RAM (4 threads). The *Benchmarks* workflow of the repository reruns both programs on GitHub-hosted machines.

Timing. Times are wall-clock medians of repeated runs after a warm-up. The machine is a shared virtual machine whose speed varies in bursts, so absolute times can differ by tens of percent between runs; the cross-framework comparison therefore interleaves the frameworks (Section 3.8) and also reports the minimum over all runs.

3 Results

3.1 Grover search

Grover’s algorithm [1] with M marked items among $N = 2^n$ reaches success probability $\sin^2((2k+1)\theta)$ after k iterations, where $\sin\theta = \sqrt{M/N}$; the optimal k is $\lfloor \pi/(4\theta) \rfloor$ [2]. QSO builds the oracle and diffusion operator as circuits of multi-controlled gates and simulates them on the full state vector.

Table 2. One marked item: simulated and exact success probability at the optimal iteration count, and the number of oracle queries of a classical search ((N+1)/2 on average) divided by k.

n	$N = 2^n$	k	P (simulated)	$\sin^2((2k+1)\theta)$	deviation	(N+1)/2 ÷ k
4	16	3	0.961319	0.961319	3.0×10^{-15}	2.8
6	64	6	0.996586	0.996586	9.3×10^{-15}	5.4
8	256	12	0.999947	0.999947	2.7×10^{-14}	10.7

n	N = 2^n	k	P (simulated)	$\sin^2((2k+1)\theta)$	deviation	(N+1)/2 ÷ k
10	1 024	25	0.999461	0.999461	6.8×10^{-14}	20.5
12	4 096	50	0.999945	0.999945	1.7×10^{-13}	41.0
14	16 384	100	1.000000	1.000000	3.9×10^{-13}	81.9
16	65 536	201	0.999988	0.999988	8.9×10^{-13}	163.0

Table 3. Multiple marked items (n = 10, N = 1024).

M	k	P (simulated)	$\sin^2((2k+1)\theta)$	deviation
1	25	0.999461	0.999461	6.8×10^{-14}
2	17	0.999448	0.999448	4.9×10^{-14}
4	12	0.999947	0.999947	3.3×10^{-14}
8	8	0.995620	0.995620	2.1×10^{-14}
16	6	0.996586	0.996586	1.5×10^{-14}
32	4	0.999182	0.999182	1.0×10^{-14}

The largest deviation from the exact formula over all single-target sizes (n = 2 … 16) and the multi-target cases is 8.9×10^{-13}, i.e. rounding error. The query ratio grows as $\sqrt{N}$, the quadratic speed-up of unstructured search. (Version 1 reported a success probability of $0.0754 \approx 1/13$ — the probability of guessing one of 13 states — and called the ratio of two unrelated quantities a speed-up.)

3.2 Quantum walks on Metatron's Cube

Metatron's Cube joins all 13 centres of the Fruit of Life by straight lines, so as a graph it is the complete graph K_{13} (78 edges, all degrees 12). Its symmetry group is the full symmetric group S_{13}; the geometry lives in the embedding (seven distinct line lengths). Keeping only the lines between tangent circles gives the Fruit-of-Life contact graph with 18 edges and the hexagonal symmetry group D_6. QSO computes automorphism groups exactly (individualisation–refinement with orbit–stabiliser bookkeeping):

Table 4. Automorphism group orders.

graph	group order (QSO)	reference
Metatron's Cube	6 227 020 800	13! (symmetric group S_{13})
Fruit of Life	12	12 (dihedral group D_6)
dodecahedron	120	120 (icosahedral group I_h)
Petersen	120	120 (S_5)
Heawood	336	336 (PGL(2, 7))

Continuous-time walk. For H = L (graph Laplacian) and a walker started at vertex 0 of K_{13}, the Laplacian spectrum $\{0, 13^{12}\}$ gives $P_0(t) = (145 + 24 \cos 13t)/169$. The time-averaged distribution therefore converges to 145/169 at the start vertex and 2/169 elsewhere. QSO computes limiting distributions from the eigendecomposition with degenerate eigenspaces grouped; it returns $P_0 = 0.857988165680474$ (exact: 0.857988165680473).

The mixing time M_ε of Aharonov et al. [3] is the smallest T from which on the time-averaged distribution stays within total-variation distance ε of the limit. On K_{13} that distance is $\frac{24}{169}\,|\sin(13T)/(13T)|$, which gives M_ε in closed form (evaluated on the same time grid, spacing 0.001):

Table 5. Mixing time of the walk on Metatron's Cube from vertex 0.

ε	M_ε (QSO)	M_ε (closed form)
0.05	0.172	0.172
0.02	0.417	0.417
0.01	1.092	1.092

The Fruit-of-Life contact graph mixes more slowly ($M_{0.05}$ = 1.00) and keeps 0.6670 of the probability at the centre in the limit.

Spatial search. Childs and Goldstone [4] search for a marked vertex w with $H = \gamma L - |w\rangle\langle w|$. On the complete graph K_N with $\gamma = 1/N$ the walk rotates the uniform superposition into $|w\rangle$, with success probability 1 at $T = \pi\sqrt{N}/2$. On Metatron's Cube QSO obtains P(T) = 1.000000000000000 at T = 5.663587; its rate optimiser, started from no prior knowledge, finds γ = 0.076923076 (1/13 = 0.076923077) and P = 1.000000000000. A classical random walk needs 12.0 steps on average to hit the marked vertex. Table 6 shows the √N scaling on larger complete graphs.

Table 6. Search on complete graphs: quantum evolution time against the mean classical hitting time N − 1.

N	T = π√N/2	P(T)	classical hitting time	ratio
13	5.66	1.000000000000	12.00	2.12
64	12.57	1.000000000000	63.00	5.01
256	25.13	1.000000000000	255.00	10.15
1024	50.27	1.000000000000	1023.00	20.35

On the Fruit of Life, which lacks the symmetry of K_{13}, optimal search depends on the marked vertex and does not reach probability 1:

Table 7. Search on the Fruit-of-Life contact graph (rate and time optimised, Laplacian generator).

marked vertex	γ	T	P(T)	T / P(T)	classical hitting time
centre	0.3147	5.46	0.5118	10.68	5.50
inner ring	0.4112	26.85	0.5287	50.78	11.63
outer ring	1.0841	18.45	0.8451	21.83	46.55

3.3 QAOA for MaxCut

QAOA [5] alternates $e^{-i\gamma C}$ for the cut operator $C = \sum_{(u,v)\in E}(1 - Z_u Z_v)/2$ with the mixer $e^{-i\beta\sum_q X_q}$ and maximises $\langle C\rangle$. QSO simulates it on the full 2^n-dimensional state, with exact gradients and layer-wise (INTERP) initialisation, and reports the approximation ratio $\langle C\rangle/C_{\max}$ with $C_{\max}$ found by enumeration.

Table 8. QAOA against analytic and published values.

instance	ratio (QSO)	reference	source
ring C_{10}, p = 1	0.7500000000	0.7500000000	[6]
ring C_{10}, p = 2	0.8333333333	0.8333333333	[6]
ring C_{10}, p = 3	0.8750000000	0.8750000000	[6]
ring C_{10}, p = 4	0.9000000000	0.9000000000	[6]
cube Q_3, p = 1	0.692450	0.6925	[5]
Heawood graph, p = 2	0.755906	0.7559	[8]

On the ring the optimum (2p+1)/(2p+2) is proven [6]. For cubic graphs in which no edge lies on a cycle of length at most 2p+1, every edge sees the same tree-like neighbourhood, so the optimal ratio is a universal constant: 0.6925 for p = 1 [5] and 0.7559 for p = 2 [8]. The 3-cube Q_3 (girth 4) and the Heawood graph (girth 6; bipartite, so $C_{\max} = |E| = 21$) satisfy the conditions. In addition, the one-layer expected cut, summed over edges, matches the closed form of Wang et al. [7] to 10^{-12} on graphs with triangles, varying degrees and random structure (part of the test-suite).

Table 9. QAOA on Metatron's Cube (maximum cut 42), the Petersen graph and random cubic graphs (mean ± standard deviation over 10 instances), with the probability of measuring an optimal cut.

p	Metatron's Cube: ratio / P(opt)	Petersen: ratio / P(opt)	random cubic, n = 14: ratio
1	0.992818 / 0.8970	0.8656 / 0.1682	0.7826 ± 0.0262
2	0.999098 / 0.9956	0.9254 / 0.4490	0.8559 ± 0.0256

p	Metatron's Cube: ratio / P(opt)	Petersen: ratio / P(opt)	random cubic, n = 14: ratio
3	0.999971 / 0.9999	0.9707 / 0.7626	0.8999 ± 0.0242
4	—	0.9947 / 0.9767	0.9240 ± 0.0209

3.4 Variational quantum eigensolver

For H_2 in the STO-3G basis at 0.735 Å (two-qubit parity-mapped Hamiltonian), VQE [9] with a one-layer real-amplitude ansatz reaches E = -1.1373060358 Ha including nuclear repulsion; exact diagonalisation of the same Hamiltonian gives -1.1373060358 Ha, the full-CI energy of the basis (−1.137306 Ha [10]).

For 8-qubit spin chains the reference is the ground-state energy from Lanczos diagonalisation. Table 10 shows how the error depends on the ansatz and its depth (four seeded restarts, L-BFGS with adjoint gradients). It is not monotone in the depth, because the optimiser can end in a local minimum; these are the values the software produces, not selected results.

Table 10. VQE on the open transverse-field Ising chain at the critical point (TFIM, J = h = 1) and the XXZ chain (J = 1, Δ = 0.5). RealAmplitudes uses RY rotations, EfficientSU2 RY and RZ rotations, both with a CX chain.

model	ansatz	layers	parameters	E (VQE)	E_0 (Lanczos)	relative error
TFIM	RealAmplitudes	1	16	-9.734201	-9.837951	1.1×10^{-2}
TFIM	RealAmplitudes	2	24	-9.824091	-9.837951	1.4×10^{-3}
TFIM	RealAmplitudes	3	32	-9.826602	-9.837951	1.2×10^{-3}
TFIM	RealAmplitudes	4	40	-9.835053	-9.837951	2.9×10^{-4}
TFIM	EfficientSU2	1	32	-9.662652	-9.837951	1.8×10^{-2}
TFIM	EfficientSU2	2	48	-9.679333	-9.837951	1.6×10^{-2}
TFIM	EfficientSU2	3	64	-9.826953	-9.837951	1.1×10^{-3}
TFIM	EfficientSU2	4	80	-9.743439	-9.837951	9.6×10^{-3}
XXZ	RealAmplitudes	1	16	-10.495361	-11.372996	7.7×10^{-2}
XXZ	RealAmplitudes	2	24	-10.742993	-11.372996	5.5×10^{-2}
XXZ	RealAmplitudes	3	32	-10.968102	-11.372996	3.6×10^{-2}
XXZ	RealAmplitudes	4	40	-11.242196	-11.372996	1.2×10^{-2}
XXZ	EfficientSU2	1	32	-10.495361	-11.372996	7.7×10^{-2}
XXZ	EfficientSU2	2	48	-10.772411	-11.372996	5.3×10^{-2}
XXZ	EfficientSU2	3	64	-11.185740	-11.372996	1.6×10^{-2}
XXZ	EfficientSU2	4	80	-11.349845	-11.372996	2.0×10^{-3}

3.5 Variational classifier

The classifier uses data re-uploading [11]: 2 qubits, 4 layers, 16 parameters, output $p(y = 1|x) = (1 - \langle Z_0 \rangle)/2$, trained on the cross-entropy with exact gradients. Each dataset has 80 training and 80 test points; the table reports mean ± standard deviation over 5 seeds and compares with logistic regression on the raw features and on quadratic features.

Table 11. Accuracy of the variational classifier and classical baselines.

dataset	VQC train	VQC test	logistic (linear) test	logistic (quadratic) test
blobs	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
circle	0.965 ± 0.016	0.922 ± 0.049	0.565 ± 0.116	1.000 ± 0.000
moons	0.988 ± 0.009	0.967 ± 0.023	0.857 ± 0.046	0.863 ± 0.028

The variational model separates the non-linear data sets (circle, moons) where linear logistic regression cannot, but a classical model with the right features does as well or better. The benchmark therefore shows a correct and reproducible training pipeline, not an advantage.

3.6 Graph classification with quantum layers

Version 1 reported a quantum machine-learning experiment on graphs. QSO now provides a graph classifier in which every input graph G defines its own cut operator C_G: p shared, trainable layers $e^{-i\beta\sum_q X_q}\,e^{-i\gamma C_G}$ act on $|+\rangle^{\otimes n}$, and $p(y=1 \mid G) = \sigma(a\,(\langle C_G\rangle/|E| - \tfrac{1}{2}) + b)$. Because the evolution is local, after p layers each edge's contribution depends only on the subgraph within distance p of the edge (the QAOA light cone). This gives a falsifiable prediction: one layer can detect triangles, but it cannot distinguish triangle-free cubic graphs with and without 4-cycles, because their radius-1 neighbourhoods are identical; two layers can.

The data are random cubic graphs (20 graphs per class in each of the training and test sets, 5 seeds). Simple degree statistics are identical for all cubic graphs, so a classifier using them cannot exceed 50 %; counting triangles or 4-cycles classifies perfectly.

Table 12. Graph classification of random cubic graphs (mean ± standard deviation over seeds; the 4-cycle task uses triangle-free graphs only).

task	nodes	layers	train accuracy	test accuracy	training time
triangle present	12	1	1.000 ± 0.000	1.000 ± 0.000	1.16 s
triangle present	12	2	0.995 ± 0.011	0.980 ± 0.033	4.33 s
4-cycle present	14	1	0.500 ± 0.000	0.500 ± 0.000	158 ms
4-cycle present	14	2	0.930 ± 0.021	0.965 ± 0.022	14.37 s

The one-layer model is at exactly 50 % on the 4-cycle task, as predicted; two layers reach high accuracy. Cubic graphs with girth ≥ 5 on 14 vertices are rare: each training set of 20 such graphs contains only 7–8 distinct graphs (distinct adjacency spectra), so training and test sets share isomorphic graphs. The result demonstrates the light cone and the training procedure, not generalisation to unseen structures.

3.7 Boson sampling

n single photons entering an m-mode interferometer U with input occupation S leave in occupation pattern T with probability $|\operatorname{perm}(U_{T,S})|^2/\prod_i t_i!$ if they are indistinguishable and $\operatorname{perm}(|U_{T,S}|^2)/\prod_i t_i!$ if they are distinguishable, where $U_{T,S}$ repeats row i of U t_i times and takes the columns of the occupied input modes [12]. QSO computes permanents with Glynn's formula in Gray-code order ($O(n\cdot 2^n)$) [14], draws Haar-random interferometers [16] and samples exactly with the algorithm of Clifford and Clifford [13].

Permanents agree with the defining sum over permutations to a relative 2.6×10^{-15} ($n \le 8$) and with The Walrus to 2.8×10^{-14} ($n \le 12$).

Hong–Ou–Mandel effect [15]. Two photons on a 50:50 beam splitter leave through different ports with probability 6.2×10^{-33} if indistinguishable and 0.500 if distinguishable: visibility 1.000000.

Bunching. If all n photons leave through one mode, the submatrix has rank one, so $P_{\text{boson}}/P_{\text{dist}} = n!$ for every interferometer:

Table 13. Full-bunching enhancement in Haar-random 8-mode interferometers.

photons n	$P_{\text{boson}}/P_{\text{dist}}$ (QSO)	n!
1	1	1
2	2	2
3	6	6
4	24	24
5	120	120
6	720	720

Exact sampling. For 4 photons in 8 modes (330 output patterns), the exact distribution sums to 1 within 2.0×10^{-15}. 200000 samples have total-variation distance 0.0133 from it; the expected distance for a perfect sampler with this

number of samples is 0.0137. Pearson's χ^2 is 310.4 over 328 cells with expected count ≥ 5, consistent with the χ^2 distribution of that many degrees of freedom.

Quantum versus classical statistics. The distance between the bosonic and the distinguishable-particle distributions grows with the photon number:

Table 14. Mean total-variation distance between indistinguishable and distinguishable photons in $m = n^2$ modes (10 Haar-random interferometers).

photons	modes	output patterns	TVD(boson, distinguishable)
2	4	10	0.264 ± 0.100
3	9	165	0.333 ± 0.030
4	16	3 876	0.381 ± 0.012

Table 15. Cost of one exact sample of n photons in $m = n^2$ modes (permanent timings are compared with The Walrus in Table 20).

photons n	modes $m = n^2$	time per exact sample
4	16	1.3 µs
6	36	3.4 µs
8	64	12.0 µs
10	100	42.5 µs
12	144	665.7 µs
14	196	2.97 ms

3.8 Comparison with other frameworks

Version 1 listed scores for Qiskit, Cirq, PennyLane and ProjectQ that were constants in the code. The comparison below runs the actual packages (Qiskit Aer 0.18.0, Qulacs 0.6.14 [18], Cirq 1.7.0, PennyLane Lightning 0.45.0 [19], The Walrus 0.22.0 [20]). ProjectQ is not included: its last release (0.8.0, 2022) does not build on current Python.

Correctness. 20 random 8-qubit circuits with 60 gates each (H, X, Y, Z, S, T, RX, RY, RZ, CX, CZ, SWAP) and a two-layer 10-qubit circuit were simulated by every package; the final states agree with QSO to rounding error:

Table 16. Largest amplitude difference to QSO.

framework	largest amplitude difference to QSO
Qiskit Aer	1.0×10^{-15}
Qulacs	2.4×10^{-15}
Cirq	3.4×10^{-16}
PennyLane Lightning	2.1×10^{-15}

Speed. Every framework ran with its default multi-threading. Each configuration ran in a fresh process; 5 rounds were interleaved with a rotating order. Entries are medians, with minima in parentheses. QSO and Qulacs include the allocation of the state; Qiskit Aer includes copying the state to Python; PennyLane includes building the circuit on each call; Cirq uses its built-in Python simulator (qsim was not tested).

Table 17. One layer (H and RZ on every qubit, CX chain).

framework	n = 16	n = 20	n = 22	n = 24
QSO	2.43 ms (2.14 ms)	16 ms (12.6 ms)	80.1 ms (68.9 ms)	552 ms (508 ms)
Qulacs	1.21 ms (698.8 µs)	16.3 ms (12.1 ms)	98.4 ms (78.8 ms)	665 ms (595 ms)
Qiskit Aer	5.18 ms (4.07 ms)	45.7 ms (37 ms)	212 ms (173 ms)	882 ms (791 ms)
PennyLane Lightning	3.76 ms (3.07 ms)	45.4 ms (42 ms)	195 ms (178 ms)	2.13 s (1.65 s)
Cirq	5.37 ms (4.53 ms)	41.5 ms (33.3 ms)	—	—

Table 18. Ten layers.

framework	n = 20, 10 layers	n = 22, 10 layers
QSO	154 ms (143 ms)	593 ms (576 ms)
Qulacs	156 ms (142 ms)	747 ms (681 ms)
Qiskit Aer	413 ms (361 ms)	1.93 s (1.73 s)
PennyLane Lightning	412 ms (394 ms)	1.73 s (1.63 s)

Table 19. Energy and gradient of a 10-qubit transverse-field Ising model with a 100-parameter ansatz.

method	value + gradient	largest gradient difference to QSO
QSO, adjoint	532.1 µs (441.5 µs)	reference
Qulacs, backprop	2.09 ms (1.72 ms)	5.3×10^{-16}
PennyLane Lightning, adjoint	19 ms (17.1 ms)	1.7×10^{-15}

From 20 qubits on, QSO is as fast as Qulacs or faster and two to four times faster than Qiskit Aer and PennyLane Lightning on this machine. On 16 qubits, where one gate takes only tens of microseconds, Qulacs is two to three times faster; at that size the cost of distributing work over the threads dominates, and QSO's handling of it is a topic for future work. The adjoint gradient is the fastest of the three methods compared; PennyLane's adjoint method is dominated by Python overhead at this size. Two QSO kernels (H and CX) were optimised during this work after a first comparison had shown a 1.2–1.7× gap to Qulacs on large registers; all numbers above are for the optimised code.

Table 20. Permanents: QSO against The Walrus (median time per permanent of a random complex n × n matrix).

n	QSO	The Walrus	ratio
10	13.2 µs	40.3 µs	3.0
14	263.7 µs	526.1 µs	2.0
18	8.02 ms	11.9 ms	1.5
20	16 ms	51.7 ms	3.2
22	37.1 ms	208 ms	5.6
24	144 ms	877 ms	6.1

3.9 Test-suite

Version 1 reported "integration benchmarks" with a compatibility score of 1.0. Their role is taken by the test-suite: 177 Rust tests (unit, validation, robustness, property and documentation tests) and 182 Python tests run on Linux, macOS and Windows, with the minimum supported Rust version and on Python 3.9 and 3.13, for every change. Releases are built and tested on all supported platforms before they are published.

4 Simulator performance

Table 21. One layer of the benchmark circuit in QSO (Rust, no Python).

qubits	gates	time per layer	ns per gate·amplitude
10	29	14.7 µs	0.494
14	41	313.7 µs	0.467
18	53	5.15 ms	0.370
20	59	14.9 ms	0.241
22	65	71.7 ms	0.263
24	71	476 ms	0.399

Registers below 2^{16} amplitudes run on one thread; the cost per gate and amplitude falls up to 22 qubits as all threads take part and rises again at 24 qubits, where one state vector takes 256 MB. A full gradient of the 100-parameter ansatz of Table 19 takes 501.7 µs by adjoint differentiation, 25.5 ms by parameter shift and 22.9 ms by finite differences.

5 Limitations

- All results are classical simulations on one machine with four threads; nothing here is measured on quantum hardware, and timings on other machines will differ.
- Problem sizes are those a classical simulator handles quickly. The benchmarks validate correctness and characterise performance; they do not claim a quantum advantage.
- The graph-classification task is small and partly memorised (Section 3.6); the variational classifier does not beat classical baselines with suitable features (Section 3.5).
- Other frameworks were used with default settings; tuned configurations (for example Qulacs with gate fusion, Cirq with qsim) may be faster.

6 Conclusion

QSO 1.1.0 reproduces every analytic and published reference used here to within rounding or statistical error, agrees with four independent simulators and a permanent library to machine precision, and is competitive in speed with established simulators. All code, data and the scripts that produce this note are public, so every number can be regenerated and checked.

Data and code availability

Source code, data and the programs that produce this note: https://github.com/LashSesh/qso (Apache-2.0). Packages: `pip install metatron-qso`, `cargo add metatron-qso-rs`. The JSON data of this version are in `docs/technical-note/data/`.

Acknowledgements

The rewrite of QSO, the audit of version 0.1 and the benchmarks in this note were developed with the assistance of an AI coding assistant (Claude, Anthropic). The author reviewed the results and is responsible for the content.

References

1. L. K. Grover, “A fast quantum mechanical algorithm for database search,” *Proc. 28th ACM STOC*, 212–219 (1996).
2. M. Boyer, G. Brassard, P. Høyer, A. Tapp, “Tight bounds on quantum searching,” *Fortschr. Phys.* 46, 493–505 (1998).
3. D. Aharonov, A. Ambainis, J. Kempe, U. Vazirani, “Quantum walks on graphs,” *Proc. 33rd ACM STOC*, 50–59 (2001).
4. A. M. Childs, J. Goldstone, “Spatial search by quantum walk,” *Phys. Rev. A* 70, 022314 (2004).
5. E. Farhi, J. Goldstone, S. Gutmann, “A quantum approximate optimization algorithm,” arXiv:1411.4028 (2014).
6. G. B. Mbeng, R. Fazio, G. E. Santoro, “Quantum annealing: a journey through digitalization, control, and hybrid quantum variational schemes,” arXiv:1906.08948 (2019).
7. Z. Wang, S. Hadfield, Z. Jiang, E. G. Rieffel, “Quantum approximate optimization algorithm for MaxCut: a fermionic view,” *Phys. Rev. A* 97, 022304 (2018).
8. J. Wurtz, P. Love, “MaxCut quantum approximate optimization algorithm performance guarantees for p > 1,” *Phys. Rev. A* 103, 042612 (2021).
9. A. Peruzzo et al., “A variational eigenvalue solver on a photonic quantum processor,” *Nat. Commun.* 5, 4213 (2014).
10. A. Szabo, N. S. Ostlund, *Modern Quantum Chemistry*, Dover (1996).
11. A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, J. I. Latorre, “Data re-uploading for a universal quantum classifier,” *Quantum* 4, 226 (2020).
12. S. Aaronson, A. Arkhipov, “The computational complexity of linear optics,” *Theory of Computing* 9, 143–252 (2013).
13. P. Clifford, R. Clifford, “The classical complexity of boson sampling,” *Proc. 29th ACM-SIAM SODA*, 146–155 (2018).
14. D. G. Glynn, “The permanent of a square matrix,” *Eur. J. Combin.* 31, 1887–1891 (2010).

15. C. K. Hong, Z. Y. Ou, L. Mandel, “Measurement of subpicosecond time intervals between two photons by interference,” *Phys. Rev. Lett.* 59, 2044–2046 (1987).
16. F. Mezzadri, “How to generate random matrices from the classical compact groups,” *Notices Amer. Math. Soc.* 54, 592–604 (2007).
17. T. Jones, J. Gacon, “Efficient calculation of gradients in classical simulations of variational quantum algorithms,” arXiv:2009.02823 (2020).
18. Y. Suzuki et al., “Qulacs: a fast and versatile quantum circuit simulator for research purpose,” *Quantum* 5, 559 (2021).
19. V. Bergholm et al., “PennyLane: automatic differentiation of hybrid quantum-classical computations,” arXiv:1811.04968 (2018).
20. B. Gupt, J. Izaac, N. Quesada, “The Walrus: a library for the calculation of hafnians, Hermite polynomials and Gaussian boson sampling,” *J. Open Source Softw.* 4, 1705 (2019).